

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
про виконання лабораторної роботи №1
“Організація професійного Python-проєкту та Git”

Виконав:

студент 3 курсу

групи ФЕП-33с

Зінчук Остап

Викладач:

Проф. Свелеба Сергій Андрійович

Львів – 2026

Хід роботи

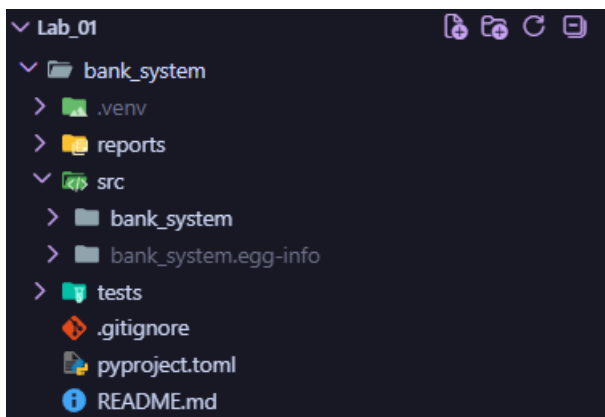
Мета: Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл `pyproject.toml`, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

Варіант: 4 (Банківська система)

Постановка задачі: Створити структурований Python-проєкт з використанням віртуального середовища (`.venv`), застосувавши підхід `src-layout`. Розділити логіку програми "Каталог автомобілів" на окремі модулі (моделі, сервіси, точка входу). Налаштувати конфігураційні файли проєкту, додати документацію та зберегти історію розробки за допомогою системи контролю версій Git щонайменше у 3 логічні коміти, після чого завантажити репозиторій на GitHub.

Структура проєкту:

Проєкт було організовано за принципом `src-layout`:



Опис створених модулів

- `__init__.py` — перетворює каталог `bank_system` на Python-пакет, зберігає номер версії.
- `config.py` — окрема конфігураційна константа: мінімально допустимий баланс (`MIN_BALANCE = 0.0`) та валюта (`CURRENCY`).
- `exceptions.py` — власні винятки (`BankError` та підкласи `NegativeAmountError`, `InsufficientFundsError`), що сигналізують про некоректні банківські операції.
- `models.py` — структура даних (`dataclass`) `Account`, яка описує сутність банківського рахунку (клієнт, номер, баланс).

- **services.py** — бізнес-логіка застосунку: поповнення, списання, пошук рахунку, обчислення загальної суми та середнього балансу, сортування, визначення найбагатшого клієнта.
- **main.py** — точка входу (entry point), де ініціалізуються демонстраційні дані та запускається інтерактивне menu-driven CLI-меню.

Код створеної програми:

Файл ***__init__.py***:

```
version = "0.1.0"
```

Файл ***config.py***

```
MIN_BALANCE: float = 0.0  
CURRENCY: str = "UAH"
```

Файл ***src/bank_system/exceptions.py***:

```
class BankError(Exception):  
    """Базовий клас для всіх помилок банківської системи."""  
  
class NegativeAmountError(BankError):  
    """Викидається, якщо сума поповнення/списання не є додатною."""  
  
class InsufficientFundsError(BankError):  
    """Викидається, якщо списання призвело б до від'ємного балансу."""  
  
class AccountNotFoundError(BankError):  
    """Викидається, якщо рахунок із заданим номером не знайдено."""
```

Файл ***src/bank_system/main.py***

```
from bank_system.config import CURRENCY  
from bank_system.exceptions import BankError  
from bank_system.models import Account  
from bank_system.services import (  
    calculate_average_balance,  
    calculate_total_funds,  
    deposit,  
    find_account,  
    find_accounts_by_client,  
    find_richest_client,  
    sort_by_balance,  
    withdraw,  
)  
  
def create_demo_accounts() -> list[Account]:  
    return [  
        Account("UA10001", "Олена Ткаченко", 15230.50),
```

```

    Account("UA100002", "Іван Петренко", 4800.00),
    Account("UA100003", "Марія Коваль", 980.75),
    Account("UA100004", "Олена Ткаченко", 250.00),
]

def print_accounts(accounts: list[Account]) -> None:
    if not accounts:
        print("Рахунків не знайдено.")
        return
    print(f"\n{'Номер':12}{'Клієнт':22}{'Баланс':>14}")
    for account in accounts:
        print(
            f"{account.account_number:12}"
            f"{account.client_name:22}"
            f"{account.balance:>10.2f} {CURRENCY}"
        )

def read_amount(prompt: str) -> float:
    while True:
        raw_value = input(prompt).strip().replace(",", ".", "")
        try:
            return float(raw_value)
        except ValueError:
            print("Некоректне число. Спробуйте ще раз.")

def print_menu() -> None:
    print("\n--- Система банківських рахунків ---")
    print("1. Показати всі рахунки")
    print("2. Поповнити рахунок")
    print("3. Зняти кошти")
    print("4. Перевірити баланс")
    print("5. Знайти рахунок(и)")
    print("6. Загальна сума коштів банку")
    print("7. Рахунки, відсортовані за балансом")
    print("8. Найбагатший клієнт")
    print("9. Вийти")

def run_menu(accounts: list[Account]) -> None:
    while True:
        print_menu()
        command = input("Оберіть команду: ").strip()

        if command == "1":
            print_accounts(accounts)

        elif command == "2":
            number = input("Номер рахунку: ").strip()
            account = find_account(accounts, number)
            if account is None:
                print("Рахунок не знайдено.")
                continue
            amount = read_amount("Сума поповнення: ")
            try:
                deposit(account, amount)
                print(f"Новий баланс: {account.balance:.2f} {CURRENCY}")
            except BankError as error:
                print(f"Помилка: {error}")

        elif command == "3":
            number = input("Номер рахунку: ").strip()
            account = find_account(accounts, number)
            if account is None:

```

```

        print("Рахунок не знайдено.")
        continue
    amount = read_amount("Сума списання: ")
    try:
        withdraw(account, amount)
        print(f"Новий баланс: {account.balance:.2f} {CURRENCY}")
    except BankError as error:
        print(f"Помилка: {error}")

elif command == "4":
    number = input("Номер рахунку: ").strip()
    account = find_account(accounts, number)
    if account is None:
        print("Рахунок не знайдено.")
    else:
        print(f"Баланс рахунку {number}: {account.balance:.2f} {CURRENCY}")

elif command == "5":
    query = input("Ім'я клієнта або номер рахунку: ").strip()
    by_number = find_account(accounts, query)
    results = (
        [by_number] if by_number is not None
        else find_accounts_by_client(accounts, query)
    )
    print_accounts(results)

elif command == "6":
    total = calculate_total_funds(accounts)
    average = calculate_average_balance(accounts)
    print(f"Загальна сума коштів: {total:.2f} {CURRENCY}")
    print(f"Середній баланс: {average:.2f} {CURRENCY}")

elif command == "7":
    print_accounts(sort_by_balance(accounts))

elif command == "8":
    richest = find_richest_client(accounts)
    if richest is not None:
        print(
            "Найбагатший клієнт:",
            richest.client_name,
            f"({richest.balance:.2f} {CURRENCY})",
        )

elif command == "9":
    print("До побачення.")
    break

else:
    print("Невідома команда. Спробуйте ще раз.")

def main() -> None:
    accounts = create_demo_accounts()
    print("Демонстраційні дані завантажено.")
    run_menu(accounts)

if __name__ == "__main__":
    main()

```

Файл `src/bank_system/models.py`:

```

from dataclasses import dataclass

@dataclass
class Account:
    account_number: str
    client_name: str
    balance: float = 0.0

    @property
    def masked_number(self) -> str:
        """Номер рахунку з прихованими першими символами (для друку)."""
        return f"***{self.account_number[-4:]} "

```

Файл *src/bank_system/services.py*

```

from bank_system.config import MIN_BALANCE
from bank_system.exceptions import InsufficientFundsError, NegativeAmountError
from bank_system.models import Account

def find_account(
    accounts: list[Account],
    account_number: str,
) -> Account | None:
    for account in accounts:
        if account.account_number == account_number:
            return account
    return None

def find_accounts_by_client(
    accounts: list[Account],
    client_name: str,
) -> list[Account]:
    query = client_name.lower()
    return [
        account
        for account in accounts
        if query in account.client_name.lower()
    ]

def deposit(account: Account, amount: float) -> None:
    if amount <= 0:
        raise NegativeAmountError(
            "Сума поповнення повинна бути додатною."
        )
    account.balance += amount

def withdraw(account: Account, amount: float) -> None:
    if amount <= 0:
        raise NegativeAmountError(
            "Сума списання повинна бути додатною."
        )
    if account.balance - amount < MIN_BALANCE:
        raise InsufficientFundsError(
            f"Недостатньо коштів на рахунок {account.account_number}: "
            f"баланс {account.balance:.2f}, потрібно {amount:.2f}."
        )
    account.balance -= amount

```

```

def check_balance(account: Account) -> float:
    return account.balance

def calculate_total_funds(accounts: list[Account]) -> float:
    return sum(account.balance for account in accounts)

def calculate_average_balance(accounts: list[Account]) -> float:
    if not accounts:
        return 0.0
    return calculate_total_funds(accounts) / len(accounts)

def sort_by_balance(
    accounts: list[Account],
    descending: bool = True,
) -> list[Account]:
    return sorted(
        accounts,
        key=lambda account: account.balance,
        reverse=descending,
    )

def find_richest_client(accounts: list[Account]) -> Account | None:
    if not accounts:
        return None
    return max(accounts, key=lambda account: account.balance)

```

Пояснення pyproject.toml:

```

[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "bank-system"
version = "0.1.0"
description = "Laboratory work 1 (variant 4): bank accounts management system"
requires-python = ">=3.11"

[project.scripts]
bank-system = "bank_system.main:main"

```

Секція [build-system] вказує інструмент збірки (setuptools). Секція [project] містить метадані пакета: назву, версію, опис, мінімальну версію Python. Секція [project.scripts] створює консольну команду bank-system для запуску програми без явного виклику python -m.

Вміст .gitignore

```

.venv/
__pycache__/
*.pyc
.pytest_cache/
.idea/
.vscode/
dist/

```

```
build/  
*.egg-info/  
.env
```

Фрагмент README:

```
# Bank System  
  
Лабораторна робота №1 з дисципліни «Професійний Python».  
Індивідуальний варіант 4 – «Система банківських рахунків».  
  
## Description  
Консольний застосунок для роботи з банківськими рахунками: поповнення,  
списання коштів, перевірка балансу, пошук рахунку та обчислення  
загальної суми коштів банку. Від'ємний баланс не допускається.  
  
## Installation & Run  
python -m venv .venv  
source .venv/bin/activate  
python -m pip install -e .  
bank-system
```

Приклад запуску:

Демонстраційні дані завантажено.

```
--- Система банківських рахунків ---  
1. Показати всі рахунки  
2. Поповнити рахунок  
3. Зняти кошти  
4. Перевірити баланс  
5. Знайти рахунок(и)  
6. Загальна сума коштів банку  
7. Рахунки, відсортовані за балансом  
8. Найбагатший клієнт  
9. Вийти  
Оберіть команду: 1
```

Номер	Клієнт	Баланс
UA100001	Олена Ткаченко	15230.50 UAH
UA100002	Іван Петренко	4800.00 UAH
UA100003	Марія Коваль	980.75 UAH
UA100004	Олена Ткаченко	250.00 UAH

```
--- Система банківських рахунків ---  
1. Показати всі рахунки  
2. Поповнити рахунок  
3. Зняти кошти  
4. Перевірити баланс  
5. Знайти рахунок(и)  
6. Загальна сума коштів банку  
7. Рахунки, відсортовані за балансом  
8. Найбагатший клієнт  
9. Вийти  
Оберіть команду: 1
```

```
--- Система банківських рахунків ---  
1. Показати всі рахунки  
2. Поповнити рахунок  
3. Зняти кошти  
4. Перевірити баланс  
5. Знайти рахунок(и)  
6. Загальна сума коштів банку  
7. Рахунки, відсортовані за балансом  
8. Найбагатший клієнт  
9. Вийти  
Оберіть команду: 3  
Номер рахунку: UA100004  
Сума списання: 500  
Помилка: Недостатньо коштів на рахунок UA100004: баланс 250.00, потрібно 500.00
```

```

--- Система банківських рахунків ---
1. Показати всі рахунки
2. Поповнити рахунок
3. Зняти кошти
4. Перевірити баланс
5. Знайти рахунок(и)
6. Загальна сума коштів банку
7. Рахунки, відсортовані за балансом
8. Найбагатший клієнт
9. Вийти
Оберіть команду: 7

Номер      Клієнт      Баланс
UA100001   Олена Ткаченко  15230.50 UAH
UA100002   Іван Петренко   4800.00 UAH
UA100003   Марія Коваль    980.75 UAH
UA100004   Олена Ткаченко  250.00 UAH

```

```

--- Система банківських рахунків ---
1. Показати всі рахунки
2. Поповнити рахунок
3. Зняти кошти
4. Перевірити баланс
5. Знайти рахунок(и)
6. Загальна сума коштів банку
7. Рахунки, відсортовані за балансом
8. Найбагатший клієнт
9. Вийти
Оберіть команду: 2
Номер рахунку: UA100004
Сума поповнення: 5000
Новий баланс: 5250.00 UAH

--- Система банківських рахунків ---
1. Показати всі рахунки
2. Поповнити рахунок
3. Зняти кошти
4. Перевірити баланс
5. Знайти рахунок(и)
6. Загальна сума коштів банку
7. Рахунки, відсортовані за балансом
8. Найбагатший клієнт
9. Вийти
Оберіть команду: 1

Номер      Клієнт      Баланс
UA100001   Олена Ткаченко  15230.50 UAH
UA100002   Іван Петренко   4800.00 UAH
UA100003   Марія Коваль    980.75 UAH
UA100004   Олена Ткаченко  5250.00 UAH

```

Результат git status:

```

(.venv) PS D:\University\year3\Професійний Python\Lab_01\bank_system> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

```

Результат git log –oneline:

```

(.venv) PS D:\University\year3\Професійний Python\Lab_01\bank_system> git log --oneline
6287fe8 (HEAD -> main, origin/main, origin/HEAD) Add configuration, custom exceptions, and refactor main application logic
d297b6f Add application entry point, CLI logic, and documentation
79baf68 Add bank account data model and business logic services
45c67c7 Create project structure and environment configuration

```

Відповіді на контрольні запитання: 1. Що таке Python-проект? Це структурований набір файлів, каталогів, модулів, залежностей та конфігурацій, об'єднаних для вирішення певного завдання.

2. Чим проєкт відрізняється від одного Python script? Скрипт зазвичай складається з одного файлу, що швидко стає незручним при збільшенні коду, тоді як проєкт розділений на логічні компоненти з налаштованою системою збірки та тестами.
3. Що таке Python module? Це звичайний файл із розширенням .py, який містить код.
4. Що таке Python package? Це каталог, що містить файли модулів і спеціальний файл `__init__.py`.
5. Для чого потрібний `__init__.py`? Щоб вказати інтерпретатору Python, що даний каталог є пакетом, вміст якого можна імпортувати.
6. Що таке src-layout? Це структура проєкту, де весь програмний код (модулі, пакети) зберігається у вкладеному каталозі `src/`.
7. Які переваги має src-layout? Це дозволяє чітко розмежувати конфігураційні файли (в корені) від самого коду, запобігає помилкам випадкового імпорту та покращує надійність тестування.
8. Для чого використовується virtual environment? Для ізоляції встановлених бібліотек та залежностей проєкту, щоб вони не конфліктували з іншими проєктами або глобальною системою.
9. Як створити .venv? За допомогою команди `python -m venv .venv`.
10. Як активувати virtual environment? У Linux/macOS командою `source .venv/bin/activate`, у Windows — `.venv\Scripts\activate`.
11. Для чого використовується pip? Це стандартний менеджер пакунків Python, який дозволяє встановлювати, оновлювати та видаляти зовнішні бібліотеки.
12. Чому рекомендується `python -m pip`? Це гарантує, що інструмент встановлення pip буде запущено саме з того середовища Python, яке зараз активоване (наприклад, з віртуального).
13. Для чого потрібний `pyproject.toml`? Файл конфігурації, який містить метадані та правила збірки сучасного Python-проєкту.
14. Що містить секція `[project]`? Основні відомості про пакет: назву, версію, короткий опис, мінімальну підтримувану версію Python.
15. Що таке editable installation? Режим встановлення через `pip install -e .`, який дозволяє редагувати сирцевий код проєкту, при цьому зміни автоматично застосовуються без необхідності перевстановлення пакета.

- 16.Для чого використовується `if __name__ == "__main__":`? Щоб функція (наприклад, `main()`) викликалась тільки за умови прямого запуску файлу, і не викликалась, якщо файл імпортується як модуль в іншому місці.
- 17.Що таке Git? Розподілена система контролю версій (DVCS) для збереження історії змін, повернення до попередніх станів та роботи в команді.
- 18.Що таке repository? Сховище проєкту під управлінням Git, яке містить усі файли, історію змін та конфігурацію системи контролю версій.
- 19.Що таке commit? Це збережений стан файлів (знімок) у репозиторії на певний момент часу із прикріпленням текстовим описом змін.
- 20.Для чого використовується `git status`? Для перегляду поточного стану відстежуваних і невідстежуваних файлів та виявлення внесених змін у робочому каталозі.
- 21.Для чого `git add`? Для додавання змінених файлів до індексу (staging area) з метою підготовки їх до створення коміту.
- 22.Для чого `git commit`? Для фіксації проіндексованих змін та створення нового запису в історії репозиторію.
- 23.Що показує `git log`? Хронологічну історію створених комітів у репозиторії.
- 24.Для чого потрібен `.gitignore`? Щоб перелічити файли або папки (наприклад, кеш, лог-файли), які Git має ігнорувати і не додавати в репозиторій.
- 25.Чому `.venv` не потрібно комітити? Тому що віртуальне середовище містить дуже багато файлів, залежить від операційної системи розробника і його легко можна відтворити з нуля командою створення.
- 26.Для чого потрібний README? Це головний документаційний файл, який описує суть проєкту та містить інструкції з його налаштування і запуску.
- 27.Що таке type hints? Це механізм анотування типів у Python (наприклад, `def main() -> None`), який підказує розробнику та редактору коду очікувані типи даних аргументів і результатів виконання функцій.
- 28.Для чого програму розділяють на modules? Це дозволяє логічно структурувати програму: відокремити введення/виведення від моделей даних та бізнес-логіки, що полегшує тестування і підтримку коду.
- 29.Що таке business logic? Це та частина програмного коду, яка виконує основні обчислення, правила обробки та перетворення даних.
- 30.Чому потрібно відокремлювати application entry point від business logic? Це дозволяє імпортувати бізнес-функції для тестування або в інші частини проєкту, не запускаючи при цьому інтерфейс самої програми

Висновок:

У ході виконання лабораторної роботи було створено повноцінний структурований Python-проект «Система банківських рахунків» (варіант 4) відповідно до сучасних стандартів організації коду. Набуто практичних навичок роботи з віртуальним середовищем, побудови src-layout, розділення програми на модулі моделей, бізнес-логіки, конфігурації та власних винятків. Реалізовано операції поповнення й списання коштів із захистом від від'ємного балансу, пошук рахунків та статистичні обчислення. Освоєно налаштування ruproject.toml, editable-встановлення пакета, а також практику роботи з Git: створення логічних комітів, ведення .gitignore та публікацію репозиторію на GitHub.